FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia

Sistemas Operativos II

Unidad 4 / Semana 9

Año: 2025

Contenidos

Interbloqueos (Deadlock): Modelo del deadlock. Tratamiento. Prevención. Evasión. Detección y recuperación.

Índice

1. Interbloqueo **3**
1.1 Ejemplo de la vida cotidiana 3
1.2 Ejemplo técnico (Sistemas Operativos) 5
1.3 Definición 6
1.4 Condiciones para el Interbloqueo 6
1.5 Clasificación de los Recursos 7
1.5.1 Recursos Apropiativos y No Apropiativos 7
1.5.2 Recursos Reutilizables y Consumibles 8
2. Estrategias para tratar el Interbloqueo **9**
2.1 Prevención del Interbloqueo 10
2.1.1 Exclusión mutua 10
2.1.2 Retención y espera 11
2.1.3 No expropiación 11
2.1.4 Espera circular 11
2.2 Predicción del Interbloqueo 12
2.3 Detección del Interbloqueo 12
2.4 Omisión del problema (estrategia del avestruz) 13
3. Recuperación del Interbloqueo **13**

1. Interbloqueo

En los sistemas operativos modernos, el **interbloqueo (o deadlock)** es uno de los principales problemas que afectan la gestión concurrente de recursos, impidiendo el progreso de los procesos afectados e impactando la eficiencia y la confiabilidad del entorno multiusuario. El objetivo de este material es brindar una comprensión estructurada de qué son los interbloqueos, cómo surgen, cómo se pueden modelar, sus condiciones necesarias y las estrategias tanto para su prevención como para su gestión y posterior solución.

Un interbloqueo ocurre cuando un conjunto de procesos queda indefinidamente bloqueado porque cada uno espera para siempre un recurso que está retenido por otro proceso del mismo conjunto. En otras palabras, ninguno puede continuar sin recibir un recurso que otro está utilizando y, a su vez, esperando liberar. Esto sucede comúnmente en ambientes donde múltiples procesos comparten recursos como impresoras, unidades de cinta, registros de datos, archivos o similares.

Esta problemática no se circunscribe solamente al ámbito de los sistemas operativos, sino que en la vida cotidiana también podemos encontrar ejemplos que nos ilustran la frecuencia con que estos casos pueden suceder.

1.1 Ejemplo de la vida cotidiana

Podemos mencionar el caso del tránsito vehicular, que nos ofrece situaciones a menudo de interbloqueos.

Veamos en detalle:

La *Figura 1* muestra una situación en la que cuatro coches han llegado aproximadamente al mismo tiempo a una intersección donde confluyen cuatro caminos.

Los cuatro cuadrantes de la intersección son los recursos que hay que controlar. En particular, si los cuatro coches desean cruzar la intersección, los requisitos de recursos son los siguientes:

- El coche 1, que viaja hacia el norte, necesita los cuadrantes a y b.

- El coche 2 necesita los cuadrantes b y c.

- El coche 3 necesita los cuadrantes c y d.

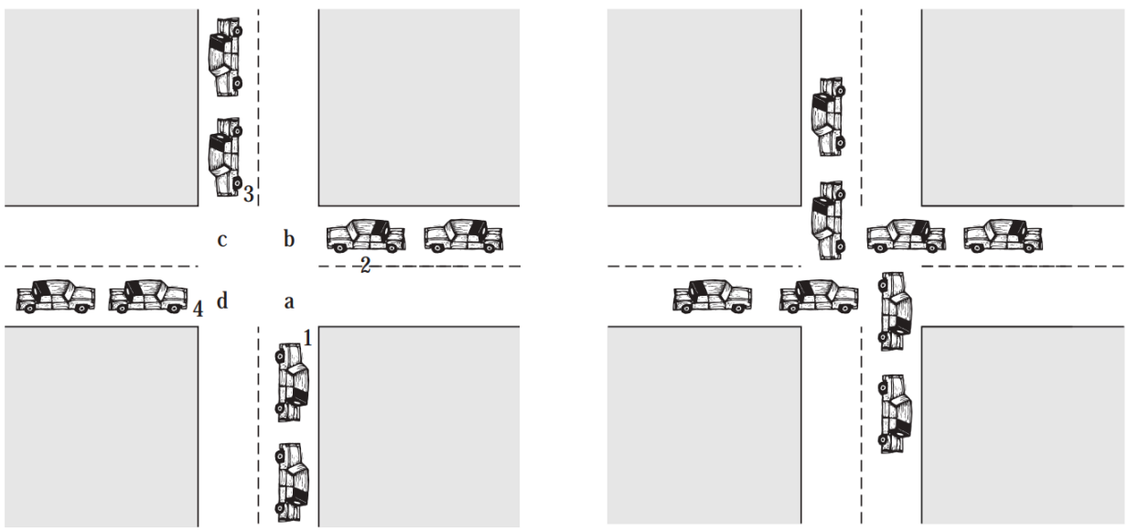
- El coche 4 necesita los cuadrantes d y a.

La norma de circulación habitual es que un coche en un cruce de cuatro caminos, sin señales ni semáforos, debería dar preferencia a otro coche que está justo a su derecha.

Esta regla funciona si hay sólo dos o tres coches en la intersección. Por ejemplo, si sólo llegan a la intersección los coches que vienen del norte y del oeste, el del norte esperará y el del oeste proseguirá.

Sin embargo, si todos los coches llegan aproximadamente al mismo tiempo, cada uno se abstendrá de cruzar la intersección, produciéndose un interbloqueo. Si los cuatro coches olvidan las normas y entran (cuidadosamente) en la intersección, cada uno posee un recurso (un cuadrante) pero no pueden continuar porque el segundo recurso requerido ya se lo ha apoderado otro coche. De nuevo, se ha producido un **interbloqueo**.

Nótese también que, debido a que cada coche tiene justo detrás otro coche, no es posible dar marcha atrás para eliminar el interbloqueo.

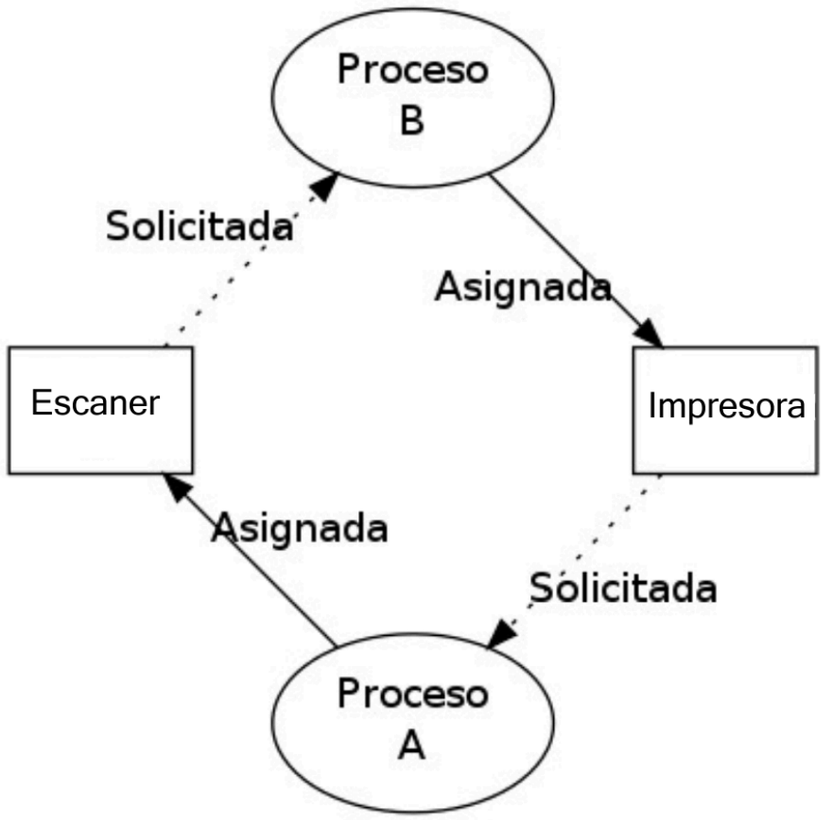


(a) Posible interbloqueo (b) Interbloqueo

Figura 1[1]

[1] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

1.2 Ejemplo técnico (Sistemas Operativos)



Imaginemos un sistema en una oficina, que cuenta con 2 recursos, una *impresora* y un *escáner* y dos usuarios (o procesos) (A y B) intentando hacer copias digitalizadas, se produce la siguiente secuencia:

1. A solicita el *escáner* y se bloquea.
2. B solicita la *impresora* y se bloquea.
3. El sistema operativo otorga el *escáner* al proceso A y lo vuelve a poner en ejecución.
4. A continúa procesando; termina su período de ejecución.
5. El sistema operativo otorga la *impresora* a B y lo vuelve a poner en ejecución.
6. B solicita el *escáner* y se bloquea.
7. El sistema operativo no tiene *escáner* disponibles y mantiene a B bloqueado; otorga el control de vuelta a A.
8. A solicita acceso a la impresora y se bloquea
9. El sistema operativo no tiene *impresora* disponible. Mantiene bloqueado tanto A como B y otorga el control de vuelta a otro proceso (o queda en espera). En este caso ni A ni B serán desbloqueados nunca.

Los interbloqueos pueden ocurrir tanto por competencia sobre recursos de hardware (dispositivos) como de software (archivos, registros, semáforos). El fenómeno es suficientemente general como para encontrarse también en sistemas distribuidos, bases de datos, redes y hasta en situaciones aparentemente triviales fuera de la informática, como vimos recientemente.

1.3 Definición

El ***interbloqueo, bloqueo mutuo o deadlock*** es una situación en la que dos o más procesos quedan bloqueados de forma permanente, cada uno esperando un *recurso* que está siendo utilizado por otro proceso del mismo conjunto. Como ninguno puede avanzar ni liberar lo que tiene, el sistema entra en un estado de espera infinita.

El **bloqueo es permanente** porque cada proceso del conjunto está bloqueado **esperando un evento** (normalmente la liberación de algún recurso requerido) que **sólo puede generar otro proceso también bloqueado** del conjunto.

1.4 Condiciones para el Interbloqueo

Deben presentarse tres condiciones de gestión para que sea posible un interbloqueo:

1. **Exclusión mutua.** Sólo un proceso puede utilizar un recurso en cada momento. Ningún proceso puede acceder a una unidad de un recurso que se ha asignado a otro proceso.

2. **Retención y espera.** Un proceso puede mantener los recursos asignados mientras espera la asignación de otros recursos.

3. **Sin expropiación.** No se puede forzar la expropiación de un recurso a un proceso que lo posee.

Aunque la presencia de estas tres condiciones permite la posibilidad de un interbloqueo, su cumplimiento no garantiza que éste ocurra.

En otras palabras, son **condiciones necesarias pero no suficientes**.

Para que el interbloqueo se manifieste efectivamente, es indispensable que se cumpla una cuarta condición adicional:

4. **Espera circular.** Existe una lista cerrada de procesos, de tal manera que cada proceso posee al menos un recurso necesitado por el siguiente proceso de la lista.

La cuarta condición es, realmente, una consecuencia potencial de las tres primeras. Es decir, si se cumplen las tres primeras condiciones, se puede producir una secuencia de eventos que conduzca a la situación de interbloqueo.

Entonces, tenemos que estas cuatro condiciones aseguran que se va a producir la situación de interbloqueo y son conocidas como *las condiciones de Coffman*.[2]

1.5 Clasificación de los Recursos

El análisis de los interbloqueos siempre parte de la noción de *recurso*. En sistemas operativos, un recurso es cualquier objeto, hardware o software, que debe adquirirse, utilizarse y liberarse a lo largo del tiempo. Esto abarca dispositivos físicos (impresoras, discos, memorias) y lógicos (registros en una base de datos, semáforos, archivos).

1.5.1 Recursos Apropiativos y No Apropiativos

La clasificación básica presentada por el autor *A.Tanenbaum*[3] distingue dos tipos:

- **Recursos apropiativos:**

  Pueden ser retirados temporalmente de un proceso y reasignados a otro sin afectar la integridad del sistema. Por ejemplo, la memoria RAM: si un proceso es removido temporalmente de memoria, al devolverle la memoria no pierde datos ni operaciones intermedias. Esto permite estrategias de recuperación y ayuda a prevenir interbloqueos.

  Ejemplo: Suponga dos procesos de 256 MB que deben imprimir algo. A obtiene la impresora pero sigue ocupando memoria; luego es removido de la memoria cuando expira su quantum. B, que ahora tiene la memoria, intenta obtener la impresora sin éxito. Si la memoria de A es apropiativa, se la puede ceder a B para que termine su impresión.

- **Recursos no apropiativos:**

  No pueden retirarse a la fuerza sin consecuencias dañinas. Por ejemplo, si un proceso está imprimiendo, no se puede despojar la impresora e interrumpir su trabajo, porque la salida impresa será incorrecta o se perderá. La mayoría de los interbloqueos relevantes involucran recursos *no apropiativos*, porque los *apropiativos* pueden resolverse precisamente mediante la redistribución forzada del recurso.

[2] Coffman, E. G., Elphick, M. J., & Shoshani, A. (1971). *System deadlocks*. ACM Computing Surveys, 3(2), 67–78**.**

[3] Tanenbaum, A. S. (2009). Sistemas operativos modernos (3ª ed., A. V. R. Elizondo, Trad.). Pearson Educación.

Tipo de recurso	Descripción	Ejemplos
Apropiativo	Puede retirarse forzosamente al proceso que lo tiene, sin consecuencias graves	Memoria, CPU
No apropiativo	No puede retirarse forzosamente sin dañar la operación o perder información	Impresoras, grabadores de CD

1.5.2 Recursos Reutilizables y Consumibles

Otros autores como Stallings emplean también una clasificación adicional:

- **Recursos reutilizables:**

  Un proceso puede usar un recurso, liberarlo y el mismo recurso puede ser usado por otro proceso posterior (ej. CPU, memoria, archivos, impresoras).

- **Recursos consumibles:**

  Una vez otorgados a un proceso, desaparecen tras el uso/consumo (señales, mensajes de comunicación entre procesos, buffers de E/S). El recurso se produce y consume, y el interbloqueo aquí suele deberse a la sincronización incorrecta en la liberación/consumo del recurso.

Tipo de recurso	Descripción	Ejemplos
Reutilizable	Puede ser empleado varias veces, no se agota por el uso y se libera para otros procesos	CPU, impresoras, memoria, semáforos
Consumible	Se crea y se destruye/consume durante el proceso	Señales, mensajes, interrupciones

Importancia frente al interbloqueo

Los interbloqueos clásicos se centran en **recursos reutilizables y no-apropiativos**. Sin embargo, existen interbloqueos por recursos consumibles, aunque el mecanismo de aparición es distinto y generalmente menos frecuente o más fácil de evitar.

La siguiente tabla resume la relación con los interbloqueos:

Categoría	Ejemplo tecnológico	¿Riesgo típico de interbloqueo?
Reutilizable, no apropiativo	Impresora, semáforo	Alto
Reutilizable, apropiativo	Memoria principal CPU Disco	Bajo
Consumible, no apropiativo	Señales de E/S, tokens de acceso, eventos de hardware	Bajo
Consumible, apropiativo	Mensajes de red, tareas en cola, trabajos en spool	Bajo

La diferencia clave radica en la capacidad de **recuperar o no** el recurso a la fuerza, y de si el recurso puede ser empleado indefinidamente.

2. Estrategias para tratar el Interbloqueo

Existen cuatro estrategias para el tratamiento del interbloqueo, tres activas y una cuarta que es pasiva, que incluso algunos autores ni la mencionan.

- En primer lugar, se puede prevenir el interbloqueo adoptando una política que elimine alguna de las condiciones (las 4 condiciones enumeradas previamente).

- En segundo lugar, se puede predecir el interbloqueo tomando las apropiadas decisiones dinámicas basadas en el estado actual de asignación de recursos.

- En tercer lugar, se puede intentar detectar la presencia del interbloqueo (cuando se cumplen las 4 condiciones) y realizar las acciones pertinentes para recuperarse del mismo.

- Por último, se podría ignorar el problema aceptando que podría ocurrir poco frecuentemente y se resolvería en forma manual.

A continuación, abordaremos estas estrategias más en detalle.

2.1 Prevención del Interbloqueo

Esta estrategia se basa en evitar que alguna de las cuatro condiciones de Coffman se cumpla. Para ello, se pueden aplicar técnicas específicas sobre cada condición.

Estas estrategias se pueden categorizar en dos:

- **Indirectas**
  Cuando se aplican a alguna/s de las primeras tres condiciones (necesarias pero no suficientes)
- **Directas**
  Cuando se trabaja sobre la cuarta condición

2.1.1 Exclusión mutua

En general, la primera de las cuatro condiciones para que se produzca un interbloqueo (la exclusión mutua) no puede eliminarse. Si un recurso requiere acceso exclusivo, el sistema operativo debe garantizarlo.

Algunos recursos, como los archivos, permiten múltiples accesos concurrentes de lectura, pero exigen exclusividad para las operaciones de escritura. Incluso en estos casos, puede surgir un interbloqueo si varios procesos solicitan simultáneamente permiso de escritura.

Se podría eliminar si el recurso permite su uso en *modo compartido,* situación poco frecuente.

2.1.2 Retención y espera

La condición de retención y espera puede eliminarse si se establece que un proceso debe solicitar todos los recursos que necesita de forma simultánea, quedando bloqueado hasta que puedan concedérsele en su conjunto.

Sin embargo, esta estrategia presenta varias limitaciones.

- En primer lugar, un proceso podría permanecer bloqueado durante un tiempo prolongado esperando la disponibilidad total de los recursos, cuando en realidad podría avanzar con solo una parte de ellos.

- En segundo lugar, los recursos asignados anticipadamente pueden quedar inutilizados durante lapsos significativos de tiempo, impidiendo su aprovechamiento por otros procesos.

- Además, no siempre es posible que un proceso conozca de antemano todos los recursos que requerirá a lo largo de su ejecución. Esta dificultad se agrava en aplicaciones con *programación modular o estructuras multihilo,* donde sería necesario prever y coordinar todas las solicitudes de recursos en distintos niveles o módulos para realizar una petición conjunta.

2.1.3 No expropiación

Si un proceso no puede obtener un recurso, debe liberar todos los demás y volver a intentar más adelante (posible pérdida de trabajo intermedio).

Otra opción es habilitar al sistema operativo a expropiar algún proceso (de menor prioridad) que tenga en uso el recurso requerido y liberar sus recursos tomados.
Esto es aplicable para el caso de recursos que permiten guardar su estado y restaurarlo posteriormente como un procesador por ejemplo.

2.1.4 Espera circular

Imponer un cierto orden global en la solicitud de recursos (por ejemplo, siempre primero escáner -> impresora, nunca al revés). Manteniendo este orden, la espera circular no puede ocurrir.

Limitaciones:

- En muchos procesos reales no se conoce con anticipación qué recursos se necesitarán

- Puede causar subutilización y posibles situaciones de “inanición” (proceso que nunca logra obtener todos los recursos que necesita)

2.2 Predicción del Interbloqueo

Una estrategia para abordar el problema del interbloqueo, que se diferencia sutilmente de la *prevención*, es la **predicción del interbloqueo**.

Mientras que la *prevención* restringe las solicitudes de recursos para evitar que se cumpla al menos una de las cuatro condiciones necesarias para que se produzca un interbloqueo, la ***predicción* permite que dichas condiciones se mantengan**, permitiendo una mayor concurrencia, pero toma decisiones dinámicas para evitar que el sistema alcance un estado de interbloqueo.

La predicción del interbloqueo evalúa en tiempo real si una solicitud de recurso, en caso de ser concedida, podría conducir a un interbloqueo.

Para ello, requiere conocer cuál será el máximo de recursos a demandar de cada proceso de antemano, lo que permite tomar decisiones más flexibles y favorecer una mayor concurrencia que la prevención.

Existen dos técnicas principales para implementar la predicción del interbloqueo:

- Evitar iniciar un proceso si sus demandas potenciales podrían derivar en un interbloqueo.

- Denegar una solicitud adicional de recurso si su concesión podría comprometer la seguridad del sistema (algoritmo del banquero)[4]

2.3 Detección del Interbloqueo

Las estrategias anteriores son bastante conservadores y restrictivas, en esta estrategia, en cambio, el sistema permite que los interbloqueos ocurran, pero periódicamente o bajo ciertas condiciones ejecuta algoritmos de detección para identificar si existe un ciclo de espera.

Los métodos típicos son:

- **Análisis de grafos de asignación de recursos:** Se busca la presencia de ciclos. Si se detectan, hay procesos en interbloqueo.

- **Representación matricial y algoritmos similares al del banquero:** Se analizan matrices de asignación, demanda y disponibilidad para ver si las necesidades restantes pueden ser satisfechas con los recursos actuales.

[4] Dijkstra, E. W. (1965). Cooperating sequential processes. En F. Genuys (Ed.), Programming Languages (pp. 43–112). Academic Press.

Frecuencia de ejecución:

Puede variar, desde cada nueva solicitud de recurso no satisfecha hasta intervalos regulares o cuando el sistema detecta caída de rendimiento (posible síntoma de interbloqueo).

2.4 Omisión del problema (estrategia del avestruz)

Muchos sistemas operativos ampliamente conocidos y utilizados, como Linux, *UNIX* y *Windows*, optan por ignorar el problema del interbloqueo en sus configuraciones generales o de usuario.

El autor ***Andrew S. Tanembaun*** denominó a esta estrategia **el algoritmo del avestruz**[5], en referencia a la actitud del animal de esconder la cabeza en la tierra, que es un mito de la antigüedad, no real.

Esta decisión se basa en que los interbloqueos reales son poco frecuentes y las estrategias para evitarlos o recuperarlos suelen implicar un alto costo en términos de recursos del sistema.

Además, en entornos como servidores, donde un reinicio ocasional es aceptable, dichas estrategias pueden considerarse innecesarias.

En estos casos, la responsabilidad de prevenir interbloqueos recae en los desarrolladores de las aplicaciones, mientras que la recuperación, si ocurre, queda sujeta a la intervención manual del administrador del sistema.

3. Recuperación del Interbloqueo

Una vez detectado el interbloqueo, se necesita contar con una estrategia para recuperarlo. Revisaremos algunas:

- Eliminar procesos involucrados (drástico, pero garantiza liberar recursos; puede repetirse hasta romper el ciclo) y es una de las más utilizadas.

- Apropiar algún recurso de un proceso (si son apropiativos; no siempre es posible ni deseable).

- Realizar un "*rollback*" o dar marcha atrás al estado previo del proceso (requiere puntos de comprobación y es poco común fuera de bases de datos).

[5] Tanenbaum, A. S. (2009). Sistemas operativos modernos (3ª ed., A. V. R. Elizondo, Trad.). Pearson Educación.

La selección de la víctima (qué proceso matar o de dónde recuperar el recurso) puede basarse en:

- El proceso que menos trabajo haya hecho
- El que consuma más recursos
- El que tenga menor prioridad
- El que tenga mayor tiempo restante estimado

Problemas potenciales:

Pérdida de trabajo computado, inconsistencias, o incluso inanición si siempre se escoge el mismo proceso para abortar.

Tabla 6.1. Resumen de las estrategias de detección, prevención y predicción de interbloqueos en sistemas operativos [ISLO80].

Estrategia	Política de reserva de recursos	Esquemas alternativos	Principales ventajas	Principales desventajas
Prevención	Conservadora; infrautiliza recursos	Solicitud simultánea de todos los recursos	• Adecuada para procesos que realizan una sola ráfaga de actividad • No es necesaria la expropiación	• Ineficiente • Retrasa la iniciación del proceso • Los procesos deben conocer sus futuros requisitos de recursos
		Expropiación	• Conveniente cuando se aplica a recursos cuyo estado se puede guardar y restaurar fácilmente	• Expropia con más frecuencia de lo necesario
		Ordenamiento de recursos	• Es posible asegurarlo mediante comprobaciones en tiempo de compilación • No necesita cálculos en tiempo de ejecución ya que el problema se resuelve en el diseño del sistema	• Impide solicitudes graduales de recursos
Predicción	A medio camino entre la detección y la prevención	Asegura que existe al menos un camino seguro	• No es necesaria la expropiación	• El SO debe conocer los futuros requisitos de recursos de los procesos • Los procesos se pueden bloquear durante largos periodos
Detección	Muy liberal; los recursos solicitados se conceden en caso de que sea posible	Se invoca periódicamente para comprobar si hay interbloqueo	• Nunca retrasa la iniciación del proceso • Facilita la gestión en línea	• Pérdidas inherentes por expropiación

Tabla Resumen de estrategias de prevención, predicción y detección de interbloqueos[6]

[6] Stallings, W. (2005). Sistemas operativos: Aspectos internos y principios de diseño (5ª ed., J. M. Peña Sánchez et al., Trads.). Pearson Educación.



Actividad S9-A1

Realice la actividad identificada como S9-*A1* (Semana 9, Actividad 1) en la sección de actividades del campus.